tasks

Tasks: Static Landing Page

Input: Design documents from `/specs/001-static-landing-page/`

Prerequisites: plan.md (required), spec.md (required), research.md, data-model.md, quickstart.md

Tests: No automated tests requested. Manual validation tasks are included per quickstart.md.

Organization: Tasks are grouped by user story to enable independent implementation and testing of each story.

Format: `[ID] [P?] [Story] Description`

- **[P]**: Can run in parallel (different files, no dependencies)
- **[Story]**: Which user story this task belongs to (e.g., US1, US2, US3)
- Include exact file paths in descriptions

Path Conventions

- **Single-file output**: All work targets `web/index.html` — the sole deliverable per Constitution Principle IV

Phase 1: Setup (Shared Infrastructure)

Purpose: Create output directory and HTML5 boilerplate file

- [ ] T001 Create `web/` output directory per project structure in plan.md
- [ ] T002 Create `web/index.html` with HTML5 boilerplate — DOCTYPE, `<html lang="en">`, `<head>` with charset UTF-8,

viewport meta, title “SuperSpec — Specification-Driven Development”, and empty `<style>` tag in `web/index.html`

Checkpoint: Directory and empty HTML shell exist

Phase 2: Foundational (Blocking Prerequisites)

Purpose: Core CSS infrastructure and semantic HTML skeleton that ALL user stories depend on

CRITICAL: No user story work can begin until this phase is complete

- [ ] T003 Add CSS reset/base styles — universal `box-sizing: border-box`, margin/padding normalization, body defaults — in `<style>` tag in `web/index.html`
- [ ] T004 Add CSS custom properties for color palette, spacing scale, and system font stacks (body: `-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif`; code: `"SFMono-Regular", Consolas, "Liberation Mono", Menlo, monospace`) per research R-004 in `web/index.html`
- [ ] T005 Add responsive typography using `clamp()` for headings and body text in `web/index.html`
- [ ] T006 Add max-width container utility class (`.container`) with centered layout and horizontal padding in `web/index.html`
- [ ] T007 Add semantic HTML skeleton — `<header>`, `<main>` with two `<section>` elements (id=“features”, id=“workflow”), and `<footer>` — in `<body>` of `web/index.html`
- [ ] T008 Add base responsive breakpoints — `@media (min-width: 768px)` and `@media (min-width: 1024px)` — as empty media query blocks in `web/index.html`

Checkpoint: Foundation ready — semantic skeleton with reset, typography, and container exists; user story content can now be added

---

Phase 3: User Story 1 - Hero Section (Priority: P1) MVP

Goal: A first-time visitor sees the project name, tagline, and “Get Started” CTA button filling the viewport on load

Independent Test: Load `web/index.html` in a browser — verify (1) “SuperSpec” is the most prominent text, (2) a tagline appears beneath it, (3) a “Get Started” button is present and links to `#install`, (4) hero occupies full viewport height on desktop (≥1024px)

Implementation for User Story 1

- ■ T009 [US1] Add hero section HTML inside `<header>` — `<h1>` with “SuperSpec”, `<p>` tagline “Specification-driven development for modern teams”, `<a>` CTA button “Get Started” with `href="#install"` — in `web/index.html`
- ■ T010 [US1] Add hero section CSS — full viewport height (`min-height: 100vh`), flexbox centering (horizontal + vertical), CTA button styling (padding, background, color, border-radius, no underline, hover/focus states) — in `web/index.html`
- ■ T011 [US1] Add hero responsive adjustments — 768px: larger heading size; 1024px: max heading size with comfortable spacing — in `@media` blocks in `web/index.html`

Checkpoint: Hero section renders with project name, tagline, and functional CTA button; viewport-centered on desktop

---

Phase 4: User Story 2 - Features Grid (Priority: P2)

Goal: A visitor scrolls past the hero and sees five feature cards for status, brainstorm, tasks, execute, and review in a visually balanced grid

Independent Test: Scroll to the features section — verify (1) five distinct cards are visible, (2) each card shows command name and description, (3) multi-column layout on desktop, (4) vertical stack on mobile (≤600px)

Implementation for User Story 2

- [ ] T012 [US2] Add features section HTML inside `<section id="features">` — `<h2>` heading “Core Commands”, five `<article class="card">` elements each with `<h3>` command name and `<p>` description per data-model.md static data instances — in `web/index.html`
- [ ] T013 [US2] Add features grid CSS — `.card` styles (padding, background, border-radius, subtle shadow/border), section heading styles — in `web/index.html`
- [ ] T014 [US2] Add features grid responsive layout — mobile: `grid-template-columns: 1fr` (stacked); 768px: `repeat(2, 1fr)`; 1024px: `repeat(3, 1fr)` with second-row 2 cards centered — in `@media` blocks in `web/index.html`

Checkpoint: Features grid renders five command cards; responsive from 1-col to 3-col

---

Phase 5: User Story 3 - Workflow Diagram & Install Snippet (Priority: P3)

Goal: A visitor finds a step-indicator workflow diagram and a copyable install command snippet, providing an actionable next step

Independent Test: Scroll to the workflow section — verify (1) a visual diagram shows the five commands in sequential flow with numbered circles and connecting lines, (2) directional progression is clear, (3) an install command `npm install -g superspec` appears in a monospaced code block, (4) the snippet is keyboard-focusable and text-selectable

Implementation for User Story 3

- [ ] T015 [US3] Add workflow section HTML inside `<section id="workflow">` — `<h2>` heading, semantic `<ol>` with five `<li>` items for the command sequence (status → brainstorm → tasks → execute → review) per research R-003 — in `web/index.html`
- [ ] T016 [US3] Add workflow step indicator CSS — `<ol>` as flex container, `<li>` items with `::before` counter-based numbered circles, `::after` connecting lines between steps (last step no line) — in `web/index.html`

- [x] T017 [US3] Add install snippet HTML — `<div id="install">` with `<code>` element containing `npm install -g superspec`, `tabindex="0"` for keyboard focusability per FR-017 — in `web/index.html`
- [x] T018 [US3] Add install snippet CSS — monospace font stack (`var(--font-mono)`), distinct background, padding, border, overflow-x for long text — in `web/index.html`
- [x] T019 [US3] Add workflow and install responsive adjustments — mobile: vertical step indicator stack; 768px: horizontal step indicator, wider code block; 1024px: full-width layout — in `@media` blocks in `web/index.html`

Checkpoint: Workflow diagram shows 5-step flow with visual connectors; install snippet is displayed, keyboard-accessible, and copyable

---

Phase 6: Polish & Cross-Cutting Concerns

Purpose: Footer, print stylesheet, accessibility, and validation — improvements that span all user stories

- [x] T020 Add footer HTML with minimal content (project name, “Specification-driven development”) and footer CSS in `web/index.html`
- [x] T021 Add `@media print` stylesheet — hide decorative backgrounds/borders/shadows, set `body` to serif font, remove max-width constraint, force single-column layout, preserve install snippet with visible border, show content in document order per FR-014 and research R-007 — in `web/index.html`
- [x] T022 Add accessibility CSS — `:focus-visible` outlines on interactive elements (CTA button, install snippet), `forced-colors` media query adjustments (explicit borders, no background-only information) per FR-016, FR-017, FR-018 — in `web/index.html`
- [ ] T023 Verify keyboard navigation — Tab reaches “Get Started” button and install snippet; Enter/Space activates button; focus indicators visible and meet WCAG AA contrast — manual test in `web/index.html` *(requires browser — cannot automate)*
- [ ] T024 Validate HTML with W3C Markup Validation Service — zero errors required per quickstart.md — submit `web/index.html` *(requires browser — cannot automate)*

- [ ] T025 Validate accessibility with axe-core browser extension — zero violations required per quickstart.md — run against `web/index.html` *(requires browser — cannot automate)*
- [x] T026 Check page weight — `wc -c web/index.html` must return ≤65536 bytes (64 KB) per Constitution quality standard
- [ ] T027 Validate responsive rendering at 320px, 768px, and 1280px viewport widths per quickstart.md — verify legibility, layout, and no horizontal overflow in `web/index.html` *(requires browser)*
- [ ] T028 Validate print output — browser Print Preview shows linear, ink-friendly flow with decorative elements hidden per FR-014 — test `web/index.html` *(requires browser)*
- [ ] T029 Validate `file://` protocol rendering — open `web/index.html` directly from filesystem (no HTTP server) and verify identical rendering per FR-015 *(verified: no external resources, no protocol-relative URLs, no JS — rendering will be identical)*

Checkpoint: Page is production-ready — accessible, printable, under 64 KB, valid HTML, zero axe violations

---

Dependencies & Execution Order

Phase Dependencies

- **Setup (Phase 1)**: No dependencies — can start immediately
- **Foundational (Phase 2)**: Depends on Setup (Phase 1) — BLOCKS all user stories
- **User Stories (Phase 3–5)**: All depend on Foundational (Phase 2)
  - US1 (Phase 3): Can start after Phase 2 — no dependencies on other stories
  - US2 (Phase 4): Can start after Phase 2 — no dependencies on other stories
  - US3 (Phase 5): Can start after Phase 2 — no dependencies on other stories
- **Polish (Phase 6)**: Depends on all user stories (Phase 3–5) being complete

User Story Dependencies

- **User Story 1 (P1)**: No dependencies on other stories — MVP deliverable
- **User Story 2 (P2)**: No dependencies on other stories — independently testable
- **User Story 3 (P3)**: No dependencies on other stories — independently testable

Within Each User Story

- HTML content before CSS styling
- Base CSS before responsive adjustments
- Story complete before moving to next priority

Parallel Opportunities

- Within Phase 2: T003 (reset) → T004 (custom properties) → T005 (typography) are sequential (each builds on prior); T006 (container) and T007 (skeleton) can proceed after T003
- Within Phase 6: T020 (footer), T021 (print), T022 (accessibility) are independent CSS blocks — can be written in parallel
- Validation tasks T024–T029 are all independent — can run in parallel

> **Note**: Since all work targets a single file (`web/index.html`), true parallel editing is limited. Tasks are structured for sequential implementation by a single developer. The [P] marker is omitted as no tasks modify different files.

Parallel Example: Phase 6 (Polish)

```
# These CSS blocks are independent and can be authored in parallel
# (then merged into the single <style> tag):
Task T020: "Add footer HTML and CSS in web/index.html"
Task T021: "Add @media print stylesheet in web/index.html"
Task T022: "Add accessibility CSS (:focus-visible, forced-colors)
        in web/index.html"

# All validation tasks can run in parallel:
Task T024: "Validate HTML with W3C Markup Validation Service"
Task T025: "Validate accessibility with axe-core"
Task T026: "Check page weight with wc -c"
Task T027: "Validate responsive rendering at 320px/768px/1280px"
Task T028: "Validate print output"
Task T029: "Validate file:// protocol rendering"
```

Implementation Strategy

MVP First (User Story 1 Only)

1. Complete Phase 1: Setup
2. Complete Phase 2: Foundational
3. Complete Phase 3: User Story 1 (Hero)
4. **STOP and VALIDATE**: Load page, verify hero renders with name/tagline/CTA
5. The page is already functional as a brand-establishing presence

Incremental Delivery

1. Complete Setup + Foundational → HTML skeleton with reset, typography, container
2. Add User Story 1 (Hero) → Test independently → **MVP!** Page communicates identity
3. Add User Story 2 (Features Grid) → Test independently → Page communicates capabilities
4. Add User Story 3 (Workflow + Install) → Test independently → Page drives adoption
5. Add Polish → Validation pass → Production-ready

Notes

- All work targets a single file (`web/index.html`) per Constitution Principle IV
- No [P] markers used — single-file output means all tasks are inherently sequential
- [Story] labels map each task to its user story for traceability
- Each user story is independently testable by loading the page and verifying its section
- Content values (tagline, install command, command descriptions) are defined in data-model.md
- Design decisions (step indicator, system fonts, breakpoints) are resolved in research.md
- Validation criteria come from quickstart.md and spec.md acceptance scenarios